

سوالات تحلیلی

سوال ۱: برای اعمال وقفه توسط پایه های ورودی/خروجی در میکروکنترلر مورد استفاده چه تنظیمات و فعال سازی هایی باید انجام شود؟

در ابتدا باید برای peripheral وقفه را فعال کنیم. برای این کار باید بیت مورد نظر را در رجیستر GPIO interrupt enable ست کنیم. سپس باید interrupt priority را تنظیم کنیم. در نهایت باید تابع ISR خود را به interrupt vector table اضافه کنیم.

یک مثال از کد:

```
int main(void)
{
    // Enable GPIO interrupt

    GPIOA->IMR |= GPIO_Pin_0; // Enable interrupt for GPIOA Pin 0
    // Enable GPIO interrupt
    GPIOA->IS |= GPIO_Pin_0; // Configure interrupt to be triggered on edge
    // Enable GPIO interrupt
    GPIOA->IEV |= GPIO_Pin_0; // Configure interrupt to be triggered on rising edge

    // Set GPIO interrupt priority
    NVIC_SetPriority(GPIO_IRQn, 2);

    // Register GPIO interrupt handler
    NVIC_EnableIRQ(GPIO_IRQn);

    while(1)
    {
        // Main program loop
    }
}

void GPIO_IRQHandler(void)
{
    // Code to handle GPIO interrupt
}
```

کد اسلاید:

```

#include "STM32F4xx.h"
#include "switches.h"

volatile unsigned count=0;

void Init_Switch(void) {

    RCC->AHB1ENR|=RCC_AHB1ENR_GPIOAEN;

    GPIOA->PUPDR|=GPIO_PUPDR_PUPDR3_0;

    SYSCFG->EXTICR[0]&=SYSCFG_EXTICR1_EXTI3_

    EXTI->IMR |= (1<<3); //Interrupt Mask
    EXTI->FTSR|= (1<<3); //Falling trigger se

    __enable_irq();
    NVIC_SetPriority(EXTI3_IRQn,0);
    NVIC_ClearPendingIRQ(EXTI3_IRQn);
    NVIC_EnableIRQ(EXTI3_IRQn);
}

void EXTI3_IRQHandler(void) {

    //Clear the EXTI pending bits
    EXTI->PR|=(1<<3);
    NVIC_ClearPendingIRQ(EXTI3_IRQn);
}

```

سوال ۲: آیا میتوان هر پایه دلخواهی را برای اعمال وقفه خارجی استفاده کرد؟ اگر چند پایه برای وقفه خارجی فعال شوند، اولویت بندی بین این وقفه ها در صورت وقوع همزمان آنها به چه صورت تعیین میشود؟

خیر - تنها از پایه های زیر برای وقفه خارجی میتوان استفاده کرد:

I/O Pin	EXTIx	I/O Pin	EXTIx	I/O Pin	EXTIx	I/O Pin	EXTIx
PA0	EXTI0	PA1	EXTI1	PA2	EXTI2	PA3	EXTI3
PB0	EXTI0	PB1	EXTI1	PB2	EXTI2	PB3	EXTI3
PC0	EXTI0	PC1	EXTI1	PC2	EXTI2	PC3	EXTI3
PD0	EXTI0	PD1	EXTI1	PD2	EXTI2	PD3	EXTI3
PE0	EXTI0	PE1	EXTI1	PE2	EXTI2	PE3	EXTI3
PF0	EXTI0	PF1	EXTI1	PF2	EXTI2	PF3	EXTI3
PG0	EXTI0	PG1	EXTI1	PG2	EXTI2	PG3	EXTI3
PH0	EXTI0	PH1	EXTI1				

اگر چند وقفه با اولویت های یکسان اتفاق بیفتد، بسته به نوع میکروکنترلر sub priority آنهاست چک می شود (میکروکنترلر ما این فیلد را ندارد)، اگر آن هم برابر بود، وقفه ها به ترتیبی که در وکتور تیبل قرار داده شده اند، اجرا می شوند.

گزارش دستور کار:

کد:

```

1  /*
2   * EXTI.c
3   *
4   * Created on: Dec 29, 2021
5   * Author: Radwa_Saeed
6   * Edited on: Nov 25, 2023
7   * By: rfAbedi
8   */
9
10 #include "EXTI.h"
11
12 void EXTI_EnableClock(void) {
13     RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
14 }
15
16 void EXTI_INIT(unsigned int Port, int EXTx, char state) {
17     SYSCFG->EXTICR[(int) EXTx/4] |= (Port << (EXTx%4)*4);
18     EXTI->IMR |= (1UL << EXTx);
19
20     if (state == FALLING || state == FALLING_RISING) {
21         EXTI->FTSR |= (1UL << EXTx);
22     }
23     if (state == RISING || state == FALLING_RISING) {
24         EXTI->RTSR |= (1UL << EXTx);
25     }
26 }
27
28 void NVIC_ConfigIRQ(int IRQn, int priority) {
29     NVIC_SetPriority(IRQn, priority);
30     NVIC_ClearPendingIRQ(IRQn);
31     NVIC_EnableIRQ(IRQn);
32 }
33

```

```

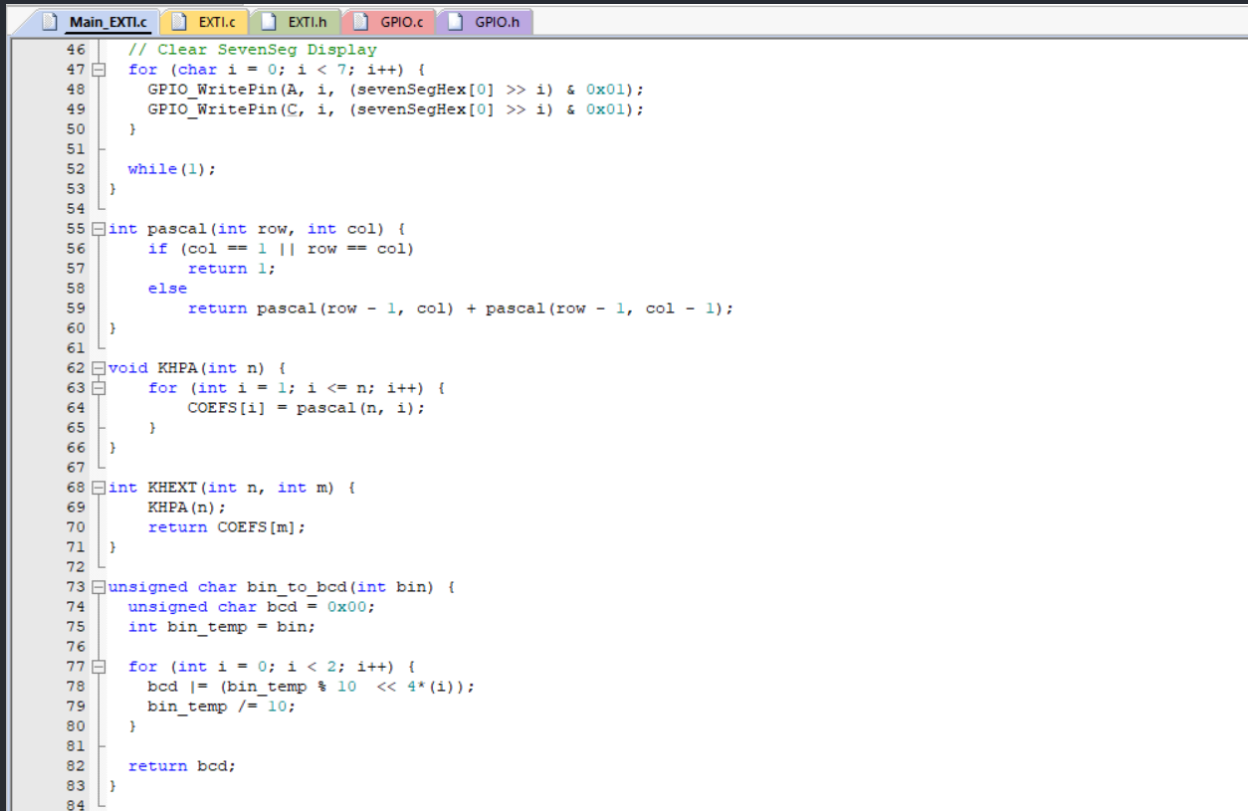
1  /*
2  * GPIO.C
3  *
4  * Created on: Nov 24, 2021
5  * Author: ADVINTIC
6  * Edited on: Nov 18, 2023
7  * By: rfAbedi
8  */
9
10 #include "GPIO.h"
11 #include "stm32f401xe.h"
12
13 static GPIO_TypeDef* GPIO_PORTS[3] = {GPIOA, GPIOB, GPIOC};
14 static unsigned long RCC_AHB1ENR_GPIOEN[3] = {RCC_AHB1ENR_GPIOAEN, RCC_AHB1ENR_GPIOBEN, RCC_AHB1ENR_GPIOCEN};
15
16
17 void GPIO_EnableClock(int Port) {
18     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOEN[Port];
19 }
20
21 void GPIO_Init(int Port, char PIN_NO, char PIN_Dir, char PIN_PuPd) {
22     switch (PIN_Dir) {
23     case (INPUT):
24         GPIO_PORTS[Port]->MODER &= ~(0x03UL << 2*PIN_NO);
25         break;
26     case (OUTPUT):
27         GPIO_PORTS[Port]->MODER |= (0x01UL << 2*PIN_NO);
28         break;
29     }
30
31     switch (PIN_PuPd) {
32     case (PULL_UP):
33         GPIO_PORTS[Port]->PUPDR |= (0x01UL << 2*PIN_NO);
34         break;
35     case (PULL_DOWN):
36         GPIO_PORTS[Port]->PUPDR |= (0x02UL << 2*PIN_NO);
37         break;
38     case (NO_PULL_UP_DOWN):
39         GPIO_PORTS[Port]->PUPDR &= ~(0x03UL << 2*PIN_NO);
40         break;
41     }
42 }
43
44 void GPIO_WritePin(int Port, char PIN_NO, char Data) {
45     if(Data) {
46         GPIO_PORTS[Port]->ODR |= (1 << PIN_NO);
47     } else {
48         GPIO_PORTS[Port]->ODR &= ~(1 << PIN_NO);
49     }
50 }
51
52 unsigned int GPIO_ReadPin(int Port, char PIN_NO) {
53     unsigned int data = 0;
54     data = (GPIO_PORTS[Port]->IDR & (1 << PIN_NO)) >> PIN_NO;
55     return data;
56 }
57

```

```

Main_EXTI.c  EXTI.c  EXTI.h  GPIO.c  GPIO.h
1  #include "GPIO.h"
2  #include "EXTI.h"
3  #include "stm32f401xe.h"
4
5  static int COEFS[10];
6  static const unsigned char sevenSegHex[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
7  static volatile int button_counter = 0;
8  static int N;
9  static int M;
10
11 int pascal(int row, int col);
12 void KHPA(int n);
13 int KHEXT(int n, int m);
14
15 unsigned char bin_to_bcd(int bin);
16
17 void EXTI15_10_IRQHandler(void);
18
19 int main(void) {
20     // Init SevenSegment
21     GPIO_EnableClock(A);
22     GPIO_EnableClock(C);
23     for (char i = 0; i < 7; i++) {
24         GPIO_Init(A, i, OUTPUT, NO_PULL_UP_DOWN);
25         GPIO_Init(C, i, OUTPUT, NO_PULL_UP_DOWN);
26     }
27
28     // Init DipSwitch
29     GPIO_EnableClock(B);
30     for (char i = 0; i < 4; i++) {
31         GPIO_Init(B, i, INPUT, NO_PULL_UP_DOWN);
32     }
33
34     // Init UserButton
35     GPIO_Init(C, 13, INPUT, PULL_UP);
36
37     // Port C EXTI
38     EXTI_EnableClock();
39     EXTI_INIT(C, EXTI13, FALLING_RISING);
40
41     //Enable Interrupt
42     __enable_irq();
43     NVIC_ConfigIRQ(EXTI15_10_IRQn, 0);

```



```

46 // Clear SevenSeg Display
47 for (char i = 0; i < 7; i++) {
48     GPIO_WritePin(A, i, (sevenSegHex[0] >> i) & 0x01);
49     GPIO_WritePin(C, i, (sevenSegHex[0] >> i) & 0x01);
50 }
51
52 while(1);
53 }
54
55 int pascal(int row, int col) {
56     if (col == 1 || row == col)
57         return 1;
58     else
59         return pascal(row - 1, col) + pascal(row - 1, col - 1);
60 }
61
62 void KHPA(int n) {
63     for (int i = 1; i <= n; i++) {
64         COEFS[i] = pascal(n, i);
65     }
66 }
67
68 int KHEXT(int n, int m) {
69     KHPA(n);
70     return COEFS[m];
71 }
72
73 unsigned char bin_to_bcd(int bin) {
74     unsigned char bcd = 0x00;
75     int bin_temp = bin;
76
77     for (int i = 0; i < 2; i++) {
78         bcd |= (bin_temp % 10 << 4*(i));
79         bin_temp /= 10;
80     }
81
82     return bcd;
83 }
84

```

```

Main_EXTI.c  EXTI.c  EXTI.h  GPIO.c  GPIO.h
81
82     return bcd;
83 }
84
85 void EXTI15_10_IRQHandler(void) {
86     EXTI->PR |= EXTI_PR_PR13;
87     NVIC_ClearPendingIRQ(EXTI15_10_IRQn);
88
89     if ((GPIO_ReadPin(C, 13) == 0) && button_counter == 0) {
90         N = 0;
91         for(char i = 0; i < 4; i++) {
92             N |= (GPIO_ReadPin(B, i) << i);
93         }
94
95         for (char i = 0; i < 7; i++) {
96             GPIO_WritePin(A, i, (sevenSegHex[bin_to_bcd(N)] >> i) & 0x01);
97         }
98
99         button_counter++;
100     } else if ((GPIO_ReadPin(C, 13) == 1) && button_counter == 1) {
101         M = 0;
102         for(char i = 0; i < 4; i++) {
103             M |= (GPIO_ReadPin(B, i) << i);
104         }
105
106         for (char i = 0; i < 7; i++) {
107             GPIO_WritePin(C, i, (sevenSegHex[bin_to_bcd(M)] >> i) & 0x01);
108         }
109
110         button_counter++;
111     } else if ((GPIO_ReadPin(C, 13) == 1) && button_counter == 2) {
112         char bcd = bin_to_bcd(KHEXT(N, M));
113
114         int digit_1 = bcd & 0x0F;
115         int digit_2 = (bcd >> 4) & 0x0F;
116
117         for (char i = 0; i < 7; i++) {
118             GPIO_WritePin(A, i, (sevenSegHex[digit_1] >> i) & 0x01);
119             GPIO_WritePin(C, i, (sevenSegHex[digit_2] >> i) & 0x01);
120         }
121
122         button_counter = 0;
123     }
124 }
125

```


تست:

